CO2 Assessment Tool 1

NAME: Kaviya V **REGISTER NO:** 192421022

Question 2:

A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.

WHITE BOX TESTING STRATEGY FOR A HEALTHCARE APPLICATION MODULE

1. Problem Understanding and Context Analysis

A healthcare application module was tested before release, but critical defects were discovered after deployment. This indicates that testing may have been performed without sufficient coverage of the module's internal logic and exceptional conditions. Testing a few normal scenarios is not enough: the module must also be checked for invalid input, duplicate records, failed database operations, and other situations that can occur in real use.

Healthcare software can manage sensitive information such as patient details, appointments, diagnoses, and prescriptions. A defect can cause incorrect records, interrupted services, or risks to patient safety. The testing approach should therefore be systematic, measurable, and based on risk. White box testing is suitable because it examines the internal code structure, decisions, conditions, and execution paths rather than checking only the visible output.

2. Conceptual Control Flow Graph

Consider a patient-registration module. It receives patient details, validates the required fields, checks whether the patient ID already exists, and then saves the record. The following simplified control flow graph shows the main execution paths.

START
↓
Read patient details
↓
Decision 1: Are required fields valid?
No → Show validation error → END
Yes ↓
Decision 2: Does the patient ID already exist?
Yes → Show duplicate warning → END
No ↓
Attempt to save patient record
Save successful?
No → Show save error / log failure → END
Yes → Confirm successful registration → END

The graph contains a start, processing steps, decisions, alternative branches, and end points. Each decision has different outcomes. Tests should execute both outcomes of each decision and verify that the correct message or action occurs. The save-failure branch is especially important because it may be missed when testing only under normal database conditions.

3. Missing Paths and Test Cases

The defects found after deployment may have occurred because the test suite covered the successful registration path but omitted alternative paths. The following tests can expose these gaps:

1. **TC1 – Valid details and new patient ID:** The record should be saved and a success confirmation displayed.

2. **TC2 – Required field is missing:** The system should reject the input, identify the missing field, and not save an incomplete record.

3. **TC3 – Invalid field format:** The system should display a clear validation message and prevent invalid data from being stored.

4. **TC4 – Duplicate patient ID:** The system should warn the user and avoid creating a duplicate record.

5. **TC5 – Database/save operation fails:** The system should report the failure safely, avoid a false success message, and preserve data consistency.

6. **TC6 – User lacks permission:** The system should block unauthorized changes and record the relevant security event where required.

These tests cover normal behavior, invalid input, duplicate data, technical failure, and access control. For a real healthcare application, the same method should be applied to high-risk features such as prescription updates, patient identity matching, and changes to medical records.

4. Critical Assessment of Coverage

Coverage type	**What it measures**	**Possible weakness in earlier testing**
Statement coverage	Whether each executable statement runs at least once.	Error-handling, duplicate-check, or save-failure statements may never run.
Branch coverage	Whether each decision outcome (true and false) is executed.	Tests may check valid fields but never check invalid fields, or test new IDs but not duplicates.
Path coverage	Whether distinct execution paths are tested.	Combinations of decisions and failure paths may be missed.

Statement coverage is useful, but it does not prove that all decision outcomes were tested. For example, a test might execute the validation statement while checking only valid input. Branch coverage gives stronger evidence by requiring both valid and invalid outcomes. Path coverage examines combinations of branches and can reveal more complex defects. However, full path coverage may be impractical when there are many decisions or loops, so critical paths should be selected according to risk. Coverage percentages should be reviewed alongside test quality; a high percentage alone does not guarantee that the software is defect-free.

5. Improved Testing Strategy

7. **Review requirements and risks.** Work with developers, testers, clinicians, and relevant stakeholders to identify expected behavior and safety-critical functions.

8. **Create the control flow graph.** Map statements, decisions, branches, and exit points for the module. Mark paths that could affect patient safety or data integrity.

9. **Design white box test cases.** Include valid input, missing fields, invalid formats, duplicate records, failed saves, and permission checks. Ensure each decision outcome is exercised.

10. **Measure coverage.** Use a code coverage tool to report statement and branch coverage. Investigate uncovered code and add tests where the missing code is relevant.

11. **Test module integration.** Check interactions with databases, authentication, patient-record services, and other connected modules. Verify that errors are handled consistently.

12. **Automate and run regression tests.** Automate repeatable tests and run them after defect fixes or code changes to confirm that existing features still work.

13. **Review defects and release readiness.** Track defects by severity, record steps to reproduce them, confirm fixes through retesting, and do not release while unacceptable critical risks remain.

6. Justification: Reliability and Risk Reduction

This approach is appropriate because it uses the control flow graph to identify exactly which decisions and paths need testing. Statement and branch coverage reports provide evidence of test adequacy, while risk-based prioritization directs effort to the functions where failures could have the greatest impact. Negative tests reveal how the module behaves when inputs or supporting services fail. Integration testing checks whether data remains consistent across connected components.

Retesting confirms that a reported defect has been corrected. Regression testing checks that the fix has not damaged previously working functions. Together, these activities reduce the chance of critical defects reaching production, improve confidence in patient data integrity, and make failures easier to detect and diagnose. They reduce risk, although no testing strategy can guarantee that all defects will be found.

7. Conclusion

The post-deployment defects suggest that the earlier testing process did not adequately cover important statements, branches, or execution paths. A conceptual control flow graph makes the missing paths visible. An improved strategy should combine white box testing, statement and branch coverage measurement, risk-based test design, integration testing, and regression testing. This structured approach strengthens reliability, protects data integrity, and reduces the likelihood of serious healthcare software failures.